

Course Logistics

- Read Chapter 30 of the textbook for this week's material on polynomials and the FFT.
- Syllabus quiz is due Sat, Jan 17. HW 1 and intro video due Fri, Jan 23.

1 Motivation: Polynomial Multiplication

We will introduce a powerful application of the divide and conquer paradigm: the Fast Fourier Transform (FFT). We will motivate it through the problem of polynomial multiplication.

Definition 1. A _____ in the variable x over an algebraic field is a representation of a function $A(x)$ as a formal sum:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

The values $a_0, a_1, \dots, a_{n-1}$ are the _____ of the polynomial.

Example 1. Polynomial Multiplication

Input: Two polynomials $A(x) = \sum_{j=0}^{n-1} a_j x^j$ and $B(x) = \sum_{j=0}^{n-1} b_j x^j$ of degree bound n .

Output: A polynomial $C(x) = A(x)B(x)$ of degree bound $2n - 1$ such that:

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j \quad \text{where} \quad c_j = \sum_{k=0}^j a_k b_{j-k}$$

An instance of this problem is multiplying $A(x) = 2 + 3x$ and $B(x) = 1 - x$.

Question 1. What is the asymptotic runtime of multiplying two polynomials of degree bound n using the standard, naive approach?

A $O(1)$

B $O(n)$

C $O(n \log n)$

D $O(n^2)$

E Other

2 Representations of Polynomials

To multiply polynomials faster, we can change how we represent them.

2.1 Coefficient Representation

We represent $A(x)$ as a vector of coefficients $a = (a_0, a_1, \dots, a_{n-1})$.

- Evaluating $A(x)$ at a given point takes _____ time.
- Multiplying two polynomials takes _____ time.

2.2 Point-Value Representation

A polynomial of degree bound n is uniquely determined by its values at n distinct points.

We can represent $A(x)$ as a set of n point-value pairs:

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$$

where $y_k = A(x_k)$ for $k = 0, 1, \dots, n-1$.

Why use point-value representation?

If we have A and B in point-value form evaluated at the *same* points $x_0, \dots, x_{2n-1}$, how long does it take to compute $C(x) = A(x)B(x)$ in point-value form?

Question 2. How many point-value pairs do we need to compute the polynomial $C(x) = A(x)B(x)$ uniquely, assuming A and B have degree bound n ?

- A** n
- B** $2n - 1$
- C** n^2
- D** $2n$

The Strategy:

- **Evaluate:** Convert A and B to point-value form at $2n$ points.
- **Point-wise Multiply:** Multiply the values in _____ time.

- **Interpolate:** Convert C back to coefficient form.

If we use arbitrary points, Evaluation and Interpolation take _____ time. We need to choose our evaluation points strategically!

3 Background Math: Complex Roots of Unity

To evaluate our polynomials efficiently, we will evaluate them at the **complex roots of unity**.

Definition 2. The n -th complex roots of unity are the n complex numbers ω such that:

Using Euler's formula, $e^{iu} = \cos(u) + i \sin(u)$, these roots are given by:

$$\omega_n^k = e^{2\pi i k/n} \quad \text{for } k = 0, 1, \dots, n-1$$

Example 2. Elementary Example: $n = 4$

The 4-th roots of unity are:

- $k = 0 : \omega_4^0 = \underline{\hspace{1cm}}$
- $k = 1 : \omega_4^1 = \underline{\hspace{1cm}}$
- $k = 2 : \omega_4^2 = \underline{\hspace{1cm}}$
- $k = 3 : \omega_4^3 = \underline{\hspace{1cm}}$

3.1 Crucial Properties for Divide and Conquer

Lemma 3.1. Cancellation Lemma: For any integers $n \geq 0, k \geq 0, d > 0$:

Lemma 3.2. Halving Lemma: If $n > 0$ is even, then the squares of the n complex n -th roots of unity are exactly the $n/2$ complex $(n/2)$ -th roots of unity.

The Halving Lemma implies that if we square our n evaluation points, we are left with only $\underline{\hspace{2cm}}$. This is perfect for divide and conquer!

4 Intuition for the Fast Fourier Transform

We want to evaluate a polynomial $A(x)$ of degree bound n at the n -th roots of unity: $1, \omega_n, \omega_n^2, \dots, \omega_n^{n-1}$. Assume n is a power of 2.

Divide We can split $A(x)$ into its even-indexed coefficients and odd-indexed coefficients:

$$\begin{aligned} A^{[0]}(x) &= a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{n/2-1} \\ A^{[1]}(x) &= a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{n/2-1} \end{aligned}$$

We can express $A(x)$ in terms of these new polynomials:

$$A(x) =$$

Conquer To evaluate $A(x)$ at the n roots of unity, we need to evaluate $A^{[0]}$ and $A^{[1]}$ at the *squares* of the n roots of unity.

By the **Halving Lemma**, these squares are exactly the $\frac{1}{2}$ -squares.

So we just need to evaluate two polynomials of degree bound $n/2$ at $n/2$ roots of unity. This is two recursive calls of size $n/2$!

Combine For $k < n/2$, we compute:

$$A(\omega_n^k) = A^{[0]}(\omega_{n/2}^k) + \omega_n^k A^{[1]}(\omega_{n/2}^k)$$

For $k \geq n/2$, we use the fact that $\omega_n^{k+n/2} = -\omega_n^k$:

$$A(\omega_n^{k+n/2}) =$$

5 The Algorithm

Here is the pseudocode for the recursive FFT algorithm, which evaluates a polynomial at the n -th roots of unity.

Algorithm 1 Recursive-FFT(a)

```
 $n = \text{length}(a)$ 
if  $n == 1$  then
    Return  $a$ 
end if
 $\omega_n = e^{2\pi i/n}$ 
 $\omega = 1$ 
 $a^{[0]} = (a_0, a_2, \dots, a_{n-2})$ 
 $a^{[1]} = (a_1, a_3, \dots, a_{n-1})$ 
 $y^{[0]} = \text{RECURSIVE-FFT}(a^{[0]})$ 
 $y^{[1]} = \text{RECURSIVE-FFT}(a^{[1]})$ 
Let  $y$  be a new array of length  $n$ 
for  $k = 0$  to  $n/2 - 1$  do
     $y_k = y_k^{[0]} + \omega \cdot y_k^{[1]}$ 
     $y_{k+n/2} = y_k^{[0]} - \omega \cdot y_k^{[1]}$ 
     $\omega = \omega \cdot \omega_n$ 
end for
Return  $y$ 
```

6 Formal Analysis

Question 3. What is the recurrence relation for the runtime $T(n)$ of the RECURSIVE-FFT algorithm?

A $T(n) = T(n/2) + \Theta(n)$

B $T(n) = 2T(n/2) + \Theta(1)$

C $T(n) = 2T(n/2) + \Theta(n)$

D $T(n) = 4T(n/2) + \Theta(n)$

Using the **Master Theorem** from Lecture 1:

- $a = \underline{\hspace{1cm}}$
- $b = \underline{\hspace{1cm}}$

- $f(n) = \underline{\hspace{2cm}}$

Since $f(n) = \Theta(n^{\log_b a})$, we are in Case 2. Thus, the runtime is:

$$T(n) = \underline{\hspace{2cm}}$$

7 Main Takeaway: The Convolution Theorem

We showed how to evaluate polynomials at roots of unity in $\Theta(n \log n)$ time. To finish polynomial multiplication, we need to interpolate the result back to coefficient form.

Interpolation It turns out that interpolating at the complex roots of unity is mathematically identical to evaluating, but using ω_n^{-1} instead of ω_n , and dividing the final results by n .

Thus, the **Inverse FFT** can also be computed in _____ time!

Fast Polynomial Multiplication Given two polynomials $A(x)$ and $B(x)$ of degree bound n :

1. **Pad:** Add zeros to A and B so they have degree bound $2n$.
2. **Evaluate (FFT):** Compute point-value representations of A and B of length $2n$.

Runtime: _____

3. **Point-wise Multiply:** Compute $y_k = A(\omega_{2n}^k)B(\omega_{2n}^k)$ for all k .

Runtime: _____

4. **Interpolate (Inverse FFT):** Convert point-values y_k back to coefficients.

Runtime: _____

Total Runtime for Polynomial Multiplication: _____.

By leveraging the **Divide and Conquer** paradigm and evaluating at mathematically symmetric points, we can bypass the naive $O(n^2)$ limit. FFT is an incredibly important algorithm used heavily in signal processing, image compression, and fast large-integer multiplication!